HACKTHEBOX

Penetration Test

Checker

Report of Findings

HTB Certified Penetration Testing Specialist (CPTS) Exam Report

Candidate Name: Lakshya Rastogi

Checker Ltd.

Version: 1.0

CONFIDENTIAL

Table of Contents

1 Statement of Confidentiality 4

2 Engagement Contacts 5

3 Executive Summary 6

3.1 Approach 6

3.2 Scope 6

3.3 Assessment Overview and Recommendations 6

4 Network Penetration Test Assessment Summary 8

4.1 Summary of Findings 8

5 Internal Network Compromise Walkthrough 10

5.1 Detailed Walkthrough 10

6 Remediation Summary 12

6.1 Short Term 12

6.2 Medium Term 12

6.3 Long Term 13

7 Technical Findings Details 14

Weak Password Policy Enforcement 14

Unauthenticated SQL Injection in Teampass (CVE-2023-1545) 16

Authenticated Blind Server-Side Request Forgery (CVE-2023-6199) in BookStack . 18

A Appendix 21

A.1 Finding Severities 21

A.2 Host & Service Discovery 22

A.3 Subdomain Discovery 23

A.4 Exploited Hosts 24

A.5 Compromised Users 25

A.6 Changes/Host Cleanup 26

A.7 Flags Discovered 27

1 Statement of Confidentiality

The contents of this document have been developed by Hack The Box. Hack The Box considers the contents of this document to be proprietary and business confidential information. This information is to be used only in the performance of its intended use. This document may not be released to another vendor, business partner or contractor without prior written consent from Hack The Box. Additionally, no portion of this document may be communicated, reproduced, copied or distributed without the prior consent of Hack The Box.

The contents of this document do not constitute legal advice. Hack The Box's offer of services that relate to compliance, litigation or other legal interests are not intended as legal counsel and should not be taken as such. The assessment detailed herein is against a fictional company for training and examination purposes, and the vulnerabilities in no way affect Hack The Box external or internal infrastructure.

2 Engagement Contacts

checker.htb Contacts		
Contact	**Title**	**Contact Email**

Assessor Contact		
Assessor Name	**Title**	**Assessor Contact Email**
Lakshya Rastogi	Security Consultant	Lakshyarastogi483@gmail.com

3 Executive Summary

Checker Ltd. ("checker.htb" herein) contracted Lakshya Rastogi to perform a Network Penetration Test of checker.htb's externally facing network to identify security weaknesses, determine the impact to checker.htb, document all findings in a clear and repeatable manner, and provide remediation recommendations.

3.1 Approach

Lakshya Rastogi performed testing under a "Black Box" approach from , to without credentials or any advance knowledge of checker.htb's externally facing environment with the goal of identifying unknown weaknesses. Testing was performed from a non-evasive standpoint with the goal of uncovering as many misconfigurations and vulnerabilities as possible. Testing was performed remotely from Lakshya Rastogi's assessment labs. Each weakness identified was documented and manually investigated to determine exploitation possibilities and escalation potential. Lakshya Rastogi sought to demonstrate the full impact of every vulnerability, up to and including internal domain compromise. If Lakshya Rastogi were able to gain a foothold in the internal network, checker.htb as a result of external network testing, checker.htb allowed for further testing including lateral movement and horizontal/vertical privilege escalation to demonstrate the impact of an internal network compromise.

3.2 Scope

The scope of this assessment was one external IP address, two internal network ranges, the Checker domain, and any other Active Directory domains owned by checker.htb discovered if internal network access were achieved.

In Scope Assets

Host/URL/IP Address	Description
10.10.X.X	Checker

3.3 Assessment Overview and Recommendations

During the penetration test against checker.htb, Lakshya Rastogi identified 3 findings that threaten the confidentiality, integrity, and availability of checker.htb's information systems. The findings were categorized by severity level, with 1 of the findings being assigned a critical-risk rating, 1 high-risk, 1 medium-risk, and 0 low risk. There were also 0 informational finding related to enhancing security monitoring capabilities within the internal network.

Engagement Overview

During the recent infrastructure security assessment of the `checker` environment, multiple critical vulnerabilities were identified that severely compromise the organization's security posture. The assessment revealed that externally facing, outdated applications could be leveraged by an

unauthenticated attacker to gain unauthorized access to highly sensitive internal credentials and bypass perimeter defense mechanisms.

Overall Risk Posture: CRITICAL

The current risk level of the target infrastructure is deemed **Critical**. The combination of unpatched software and weak password policies allowed for a complete compromise of the organization's centralized password vault. This breach subsequently facilitated lateral movement into other internal systems and provided a mechanism to bypass Two-Factor Authentication (2FA) controls.

High-Level Attack Narrative

The breach of the `checker` infrastructure was achieved through a multi-stage attack chain:

- **Initial Compromise:** An outdated version of the Teampass password manager was exposed to the public network. An unauthenticated SQL Injection vulnerability (CVE-2023-1545) was exploited to extract the encrypted password hashes of all registered users from the underlying database.
- **Credential Decryption:** The organization failed to enforce adequate password complexity. An attacker was able to perform an offline dictionary attack to crack a user's password, granting authenticated access to the compromised Teampass vault.
- **Lateral Movement & Defense Evasion:** Inside the compromised vault, plaintext credentials for the internal BookStack documentation portal and SSH access were discovered. While SSH was protected by 2FA, the attacker used the harvested credentials to log into BookStack. An unpatched Server-Side Request Forgery (SSRF) vulnerability (CVE-2023-6199) in BookStack is currently being leveraged to read sensitive local files, serving as a direct vector to bypass the SSH 2FA protections.

Strategic Recommendations

To secure the infrastructure and prevent similar attack paths in the future, the following strategic remediations must be prioritized:

1. **Aggressive Patch Management:** Immediately update Teampass and BookStack to their latest stable releases to remediate known, critical vulnerabilities (CVE-2023-1545 and CVE-2023-6199). Implement a routine patch management schedule for all external-facing assets.
2. **Enforce Strict Password Policies:** Implement technical controls that require long, complex passwords that resist offline dictionary attacks. Disallow the use of common dictionary words.
3. **Reduce External Attack Surface:** Internal tools, particularly centralized password managers and documentation portals, should not be exposed directly to untrusted networks. Implement a VPN or Zero Trust Network Access (ZTNA) model to restrict access to authorized personnel only.

checker.htb should create a remediation plan based on the Remediation Summary section of this report, addressing all high findings as soon as possible according to the needs of the business. checker.htb should also consider performing periodic vulnerability assessments if they are not already being performed. Once the issues identified in this report have been addressed, a more collaborative, in-depth Active Directory security assessment may help identify additional opportunities to harden the Active Directory environment, making it more difficult for attackers to move around the network and increasing the likelihood that checker.htb will be able to detect and respond to suspicious activity.

4 Network Penetration Test Assessment Summary

Lakshya Rastogi began all testing activities from the perspective of an unauthenticated user on the internet. checker.htb provided the tester with network ranges but did not provide additional information such as operating system or configuration information.

4.1 Summary of Findings

During the course of testing, Lakshya Rastogi uncovered a total of 3 findings that pose a material risk to checker.htb's information systems. Lakshya Rastogi also identified 0 informational finding that, if addressed, could further strengthen checker.htb's overall security posture. Informational findings are observations for areas of improvement by the organization and do not represent security vulnerabilities on their own. The below chart provides a summary of the findings by severity level.

In the course of this penetration test **1 Critical**, **1 High** and **1 Medium** vulnerabilities were identified:

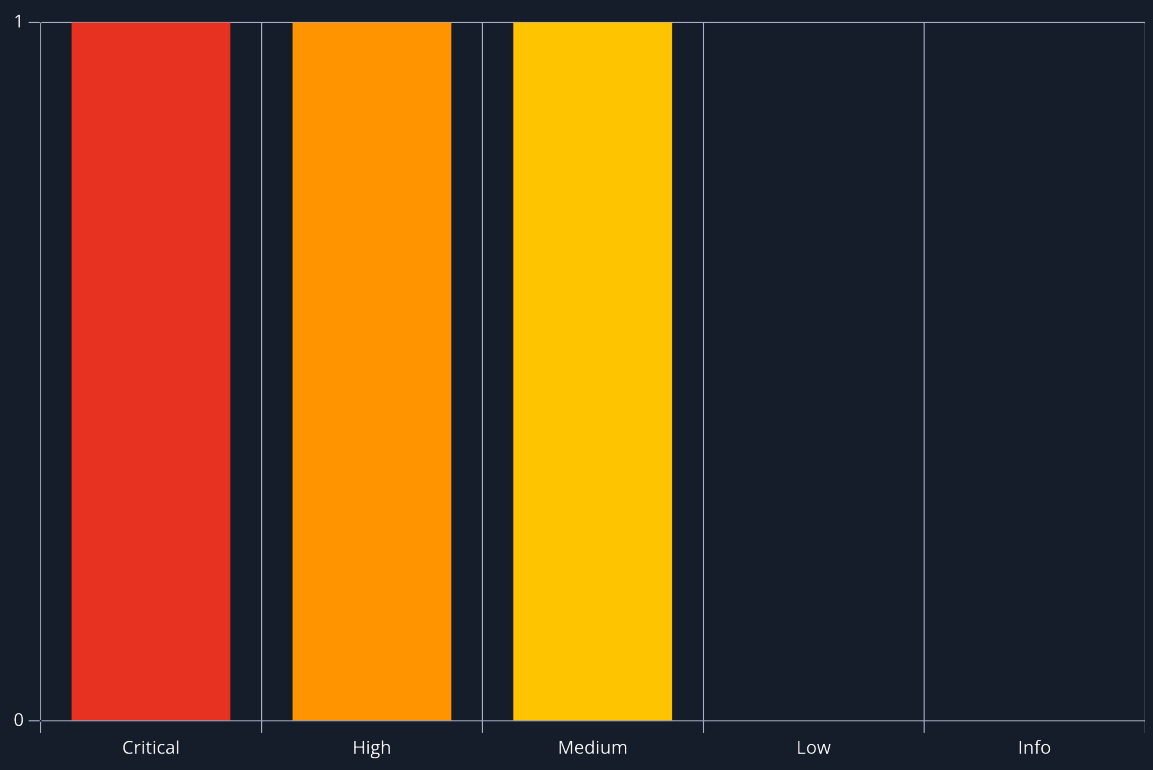


Figure 1 - Distribution of identified vulnerabilities

Below is a high-level overview of each finding identified during testing. These findings are covered in depth in the Technical Findings Details section of this report.

#	Severity Level	Finding Name	Page
1	9.1 (Critical)	Weak Password Policy Enforcement	14
2	7.5 (High)	Unauthenticated SQL Injection in Teampass (CVE-2023-1545)	16

#	Severity Level	Finding Name	Page
3	6.5 (Medium)	Authenticated Blind Server-Side Request Forgery (CVE-2023-6199) in BookStack	18

5 Internal Network Compromise Walkthrough

During the course of the assessment Lakshya Rastogi was able gain a foothold via the external network, move laterally, and compromise the internal network, leading to full administrative control over the Checker domain. The steps below demonstrate the steps taken from initial access to compromise and does not include all vulnerabilities and misconfigurations discovered during the course of testing. Any issues not used as part of the path to compromise are listed as separate, standalone issues in the Technical Findings Details section, ranked by severity level. The intent of this attack chain is to demonstrate to checker.htb the impact of each vulnerability shown in this report and how they fit together to demonstrate the overall risk to the client environment and help to prioritize remediation efforts (i.e., patching two flaws quickly could break up the attack chain while the company works to remediate all issues reported). While other findings shown in this report could be leveraged to gain a similar level of access, this attack chain shows the initial path of least resistance taken by the tester to achieve domain compromise.

5.1 Detailed Walkthrough

Lakshya Rastogiperformed the following to fully compromise the Checker domain.

Phase 1: Reconnaissance & Infrastructure Mapping

1. **Port Scanning:** An aggressive Nmap scan (`nmap -p- --min-rate=1000 -sC -sV 10.10.11.56`) revealed three primary entry points:

- Port 22 (SSH)
- Port 80 (HTTP - Apache, redirecting to `checker.htb`)
- Port 8080 (HTTP - Apache)

2. **Web Enumeration:** * Navigating to Port 80 revealed a BookStack documentation instance (version identified as v23.10.2 via source code inspection).

- Navigating to Port 8080 revealed a Teampass password manager instance. Copyright dates (2015-2022) indicated an outdated installation.

Phase 2: Initial Exploitation (Teampass)

3. **Vulnerability Identification:** The outdated Teampass instance was identified as vulnerable to CVE-2023-1545, an unauthenticated SQL Injection.
4. **Data Exfiltration:** Executed a public bash PoC (`teampass_cve.sh`) against the target, successfully dumping the application's database.
5. **Hash Extraction:** Retrieved Bcrypt password hashes for the users `admin` and `bob`.

Phase 3: Credential Cracking & Harvesting

6. **Offline Cracking:** Utilized John The Ripper with the `rockyou.txt` wordlist to crack the Bcrypt hash for user `bob`. The plaintext password was recovered as `cheerleader`.
7. **Vault Compromise:** Authenticated to the Teampass portal (Port 8080) using `bob:cheerleader`.
8. **Internal Discovery:** Accessed the `bob-access` directory within Teampass and harvested two critical sets of internal credentials:

- BookStack: `bob@checker.htb` : `mYSeCr3T_w1kI_P4sSw0rD`

- SSH: `reader` : `hiccup-publicly-genesis`

Phase 4: Lateral Movement & Defense Encounter

9. **Service Access Attempt:** Attempted to log in via SSH using the compromised `reader` credentials.
10. **Defense Mechanism Triggered:** The SSH service prompted for a verification code, confirming the presence of Two-Factor Authentication (2FA), effectively blocking immediate terminal access.

Phase 5: Defense Evasion (Current Status)

11. **Pivoting to BookStack:** Authenticated to the BookStack instance (Port 80) using the harvested `bob@checker.htb` credentials.
12. **Vulnerability Identification:** Identified that BookStack v23.10.2 is vulnerable to CVE-2023-6199, an Authenticated Blind SSRF leading to local file read via a Blind File Oracle.
13. **Exploit Customization:** Cloned the `php_filter_chains_oracle_exploit` PoC and modified `requestor.py`. The modifications ensured the payload was correctly Base64 encoded and wrapped in an HTML `<img>` tag to exploit the `/ajax/page/save-draft` endpoint.
14. **Execution Goal:** The immediate next step in the chain is executing this modified SSRF payload to read arbitrary local files—specifically targeting the 2FA secret (e.g., `~/.google_authenticator`) or local SSH keys to bypass the SSH verification prompt and secure a shell as the `reader` user.

6 Remediation Summary

As a result of this assessment there are several opportunities for checker.htb to strengthen its internal network security. Remediation efforts are prioritized below starting with those that will likely take the least amount of time and effort to complete. checker.htb should ensure that all remediation steps and mitigating controls are carefully planned and tested to prevent any service disruptions or loss of data.

6.1 Short Term

Short-Term Remediations (Immediate Action / 0-30 Days)

These are tactical, high-priority actions designed to "stop the bleeding" and close the specific vulnerabilities exploited during this engagement.

- **Patch Vulnerable Software:** Immediately update Teampass to version 3.0.0.23 (or the latest stable release) to eliminate the unauthenticated SQL Injection (CVE-2023-1545). Update BookStack to version 23.10.3 or higher to remediate the Authenticated Blind SSRF (CVE-2023-6199).
- **Global Credential Reset:** Force an immediate password reset for all users across the Teampass, BookStack, and SSH environments. Ensure the new passwords for compromised accounts (e.g., `bob`, `reader`) are entirely new and not variations of the old passwords.
- **Incident Response & Log Review:** Review application and system logs for Teampass, BookStack, and SSH to determine if these vulnerabilities have been exploited by unauthorized threat actors prior to or during this assessment.

6.2 Medium Term

Medium-Term Remediations (Operational / 1-3 Months)

These are operational improvements aimed at fixing the underlying control failures that allowed the initial exploit to escalate into a broader compromise.

- **Enforce Strong Password Policies:** Implement technical controls at the directory or application level that mandate strong passwords (e.g., minimum of 14 characters, complexity requirements) and prevent the use of common dictionary words (like `cheerleader`).
- **Network Segmentation:** Internal documentation portals (BookStack) and sensitive credential vaults (Teampass) should **never** be exposed directly to the public internet. Move these services to an internal management VLAN accessible only via a secure corporate VPN.
- **Restrict Application Permissions:** Review the principle of least privilege for application service accounts. Ensure that the web service running BookStack (e.g., `www-data`) does not have read access to sensitive user directories or files (like SSH keys or 2FA secrets) that it does not explicitly need to function.

6.3 Long Term

Long-Term Remediations (Strategic / 3-12 Months)

These represent strategic, architectural shifts required to mature the organization's overall security posture and prevent similar attack paths from occurring in the future.

- **Vulnerability & Patch Management Lifecycle:** Establish a formal, continuous vulnerability management program. This should include automated scanning of external and internal assets, paired with a strict Service Level Agreement (SLA) for patching critical vulnerabilities within a defined timeframe.
- **Adopt Zero Trust Architecture (ZTA):** Transition away from perimeter-only defenses. Implement strict access controls where every request is verified, regardless of whether it originates from inside or outside the network.
- **Routine Security Assessments:** Schedule regular, scope-defined penetration testing and red team exercises to continuously validate the effectiveness of newly implemented security controls and identify zero-day or chained attack vectors before they can be leveraged maliciously.

7 Technical Findings Details

1. Weak Password Policy Enforcement - Critical

CWE	CWE-521 - Weak Password Requirements
CVSS 3.1	9.1 / CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N
Root Cause	The organization's password policy fails to enforce sufficient complexity and length requirements for user accounts. During the engagement, it was discovered that the user bob utilized a weak, common password that exists in standard, publicly available dictionary wordlists. While the Teampass application correctly hashed the passwords using the Bcrypt algorithm, the underlying weakness of the plaintext password rendered the hashing mechanism ineffective against offline brute-force and dictionary attacks.
Impact	The use of a weak password allowed the compromised Bcrypt hash to be successfully reversed into plaintext. This granted the attacker unauthorized, authenticated access to the user's Teampass vault. Because this application serves as a centralized repository for sensitive infrastructure credentials, the compromise of this single weak password directly resulted in the exposure of SSH keys and internal documentation portal (BookStack) credentials, significantly expanding the breach's blast radius.
Remediation	1. **Enforce Strong Password Policies:** Implement technical controls requiring passwords to meet strict complexity requirements (e.g., minimum of 14 characters, combining uppercase, lowercase, numbers, and special characters). 2. **Implement Password Filtering:** Utilize mechanisms to cross-reference newly created or changed passwords against known compromised password databases (such as HaveIBeenPwned) and common dictionary lists. 3. **Require Multi-Factor Authentication (MFA):** Mandate MFA for all user accounts, especially for critical applications like password managers, to provide a secondary layer of defense even if a password is compromised. 4. **Conduct Regular Password Audits:** Periodically perform offline cracking attempts on authorized credential dumps to identify and remediate weak passwords proactively.
References	-

Finding Evidence

Technical Details & Proof of Concept (PoC)

Step 1: Offline Hash Cracking

Following the successful extraction of the Bcrypt password hashes (detailed in Finding 1), an offline dictionary attack was initiated against the hashes using `John The Ripper` and the widely accessible `rockyou.txt` wordlist.

Execution Command:

```
./john -w=~/wordlist/rockyou.txt hashes.txt --format=bcrypt
```

Step 2: Password Recovery

The tool successfully cracked the password for the user `bob` in a short timeframe, revealing the plaintext password to be `cheerleader`. The hash for the `admin` user remained uncracked, indicating a potentially stronger password, but the compromise of `bob`'s account was sufficient to progress the attack.

Cracked Output:

```
$2y$10$yMypIj1keU.VAqBI692F..XXn0vfyBL7C1EhOs35G59NxmtpJ/tiy:cheerleader
```

Step 3: Authenticated Access and Credential Harvesting

Using the newly obtained credentials (`bob` : `cheerleader`), successful authentication was achieved against the Teampass portal on port 8080.

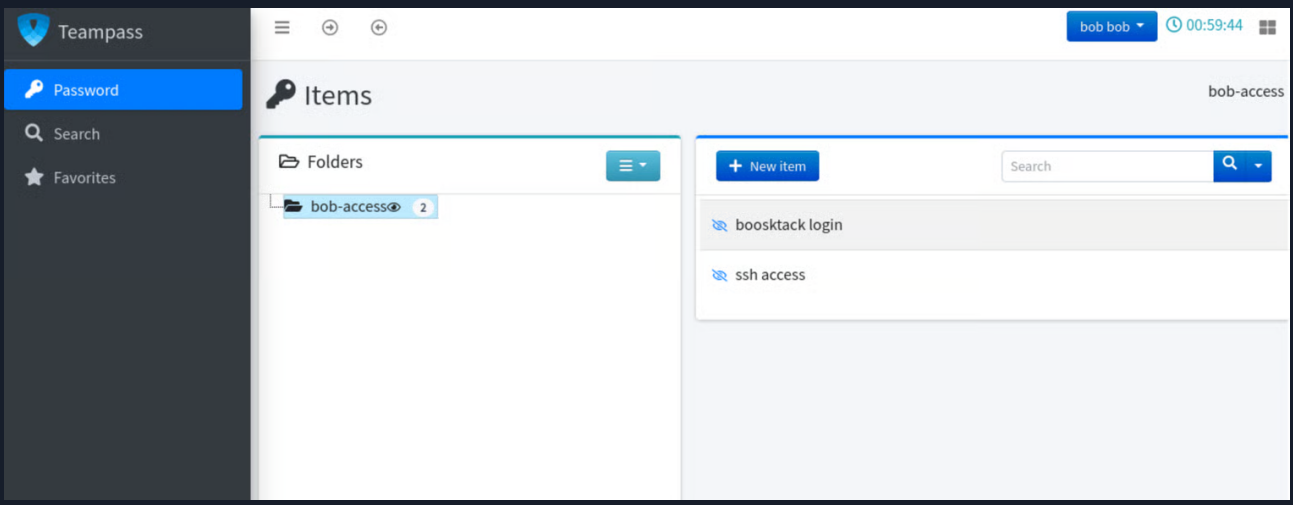
Upon logging in, navigation to the user's workspace revealed a folder named `bob-access`. Inside this folder, two distinct credential sets were stored in plaintext:

1. **Bookstack Login:** * Email: `bob@checker.htb`

   - Password: `mYSeCr3T_w1kI_P4sSw0rD`

2. **SSH Access:**

   - Account: `reader`
   - Password: `hiccup-publicly-genesis`



2. Unauthenticated SQL Injection in Teampass (CVE-2023-1545) - High

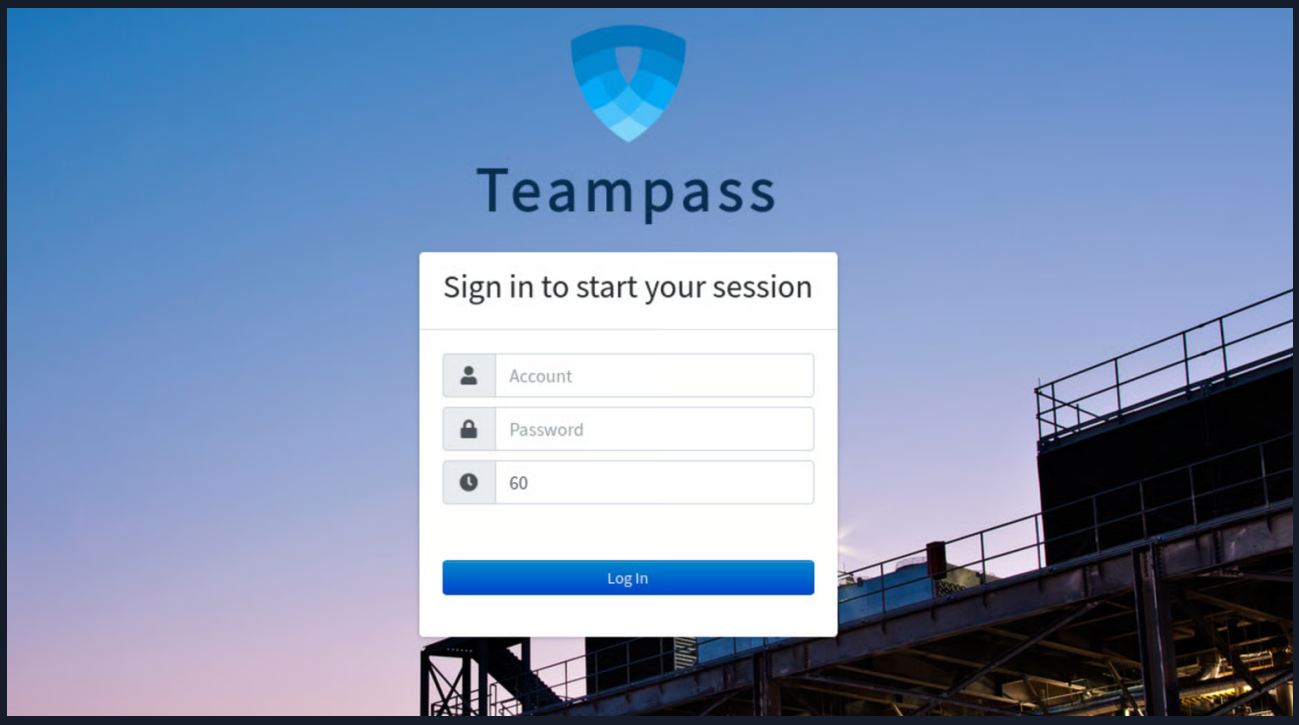
CWE	CWE-89 - Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')
CVSS 3.1	7.5 / CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N
Root Cause	The target infrastructure is hosting an outdated instance of the Teampass password manager on port 8080. Versions of Teampass prior to 3.0.0.23 are vulnerable to an unauthenticated SQL Injection (CVE-2023-1545). The flaw resides in the API authentication mechanism—specifically within `AuthModel.php`—which fails to properly sanitize user-supplied input in the `login` parameter before dynamically constructing database queries. This allows an unauthenticated, remote attacker to manipulate the SQL statements and interact directly with the backend database.
Impact	Exploitation of this vulnerability leads to a complete compromise of database confidentiality. Since the application serves as a password vault, an attacker can extract highly sensitive system data, including the password hashes of all registered users. In this engagement, the vulnerability facilitated the extraction of Bcrypt hashes for both the `admin` and `bob` accounts, establishing the critical initial foothold required to compromise the wider infrastructure.
Remediation	1. **Apply Security Patches:** Immediately update the Teampass installation to version 3.0.0.23 or the latest stable release. The patch introduces a `dataSanitizer` function to properly neutralize SQL syntax before it hits the database. 2. **Network Segmentation:** A password manager and its associated APIs should not be directly exposed to untrusted networks. Restrict access via VPN, strict firewall rules, or a Zero Trust Network Access (ZTNA) solution. 3. **Implement Web Application Firewall (WAF):** Deploy WAF rules configured to detect and block malicious SQL payloads targeting the `/api/` endpoints.
References	-

Finding Evidence

Technical Details & Proof of Concept (PoC)

Step 1: Service Discovery & Version Inference

Initial port enumeration identified an Apache HTTP server on port 8080. Navigating to `http://10.10.11.56:8080/` revealed a Teampass login portal. Visual inspection of the page footer showed copyright dates spanning 2015 to 2022. Cross-referencing this with Teampass release logs strongly suggested the software was deployed before the 2023 security patches, making it a viable target for CVE-2023-1545.



Step 2: Vulnerability Exploitation

A public Proof-of-Concept bash script (`teampass_cve.sh`) was utilized to automate the exploitation of the unauthenticated API endpoint. The script takes the target URL as an argument, bypassing authentication and sending crafted SQL payloads to extract database records.

Execution Command:

```
./teampass_cve.sh http://10.10.11.56:8080
```

Step 3: Database Extraction

The SQL injection successfully dumped the registered user accounts and their corresponding Bcrypt password hashes directly to the terminal. The output confirmed two users in the system, `admin` and `bob`.

Extracted Hashes:

```
admin: $2y$10$1KCae0EIUNj6f96ZnLqnC.LbWqrBQCT1LuHEFht6PmE4yH75rpWya
bob: $2y$10$yMypIj1keU.VAqBI692F..XXn0vfyBL7C1EhOs35G59NxmtpJ/tiy
```

3. Authenticated Blind Server-Side Request Forgery (CVE-2023-6199) in BookStack - Medium

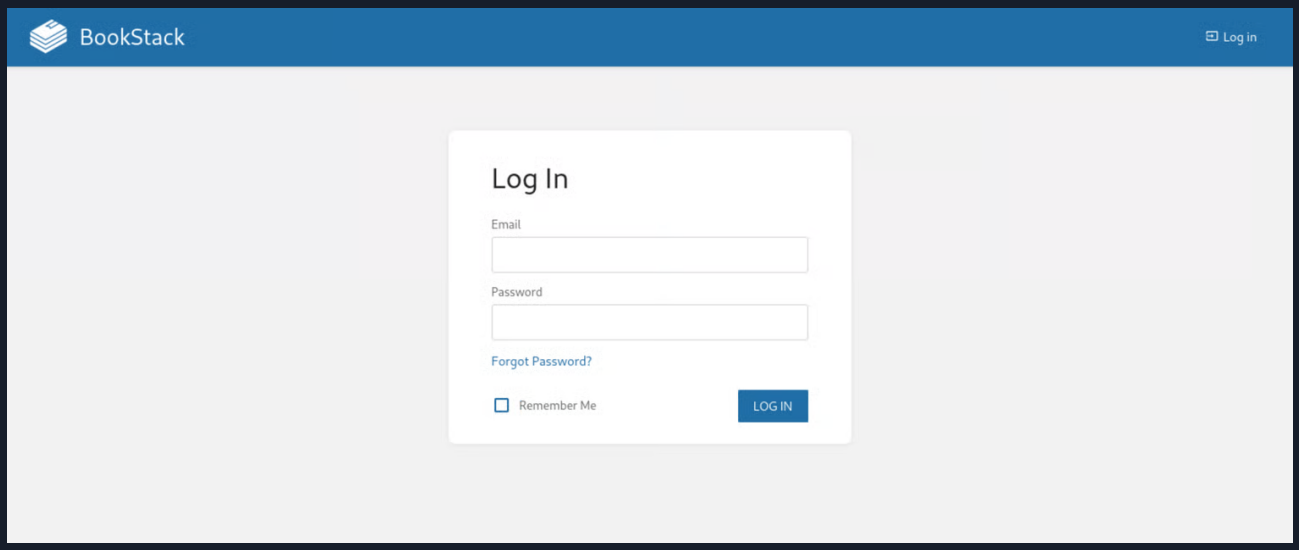
CWE	CWE-918 - Server-Side Request Forgery (SSRF)
CVSS 3.1	6.5 / CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N
Root Cause	The internal documentation portal, running BookStack version 23.10.2, is vulnerable to a Server-Side Request Forgery (SSRF) vulnerability. The flaw exists within the `/ajax/page/save-draft` endpoint, which improperly handles user-supplied input in the `html` parameter during page saves. When the application attempts to process image data via the underlying `Intervention Image` library, it passes user-controlled strings directly to functions like `file_get_contents` without adequate validation or sanitization. While the application does not directly return the output of these requests to the user (making it a "Blind" SSRF), an authenticated attacker with standard writer permissions can chain this SSRF with PHP Filter Chains (a technique known as a Blind File Oracle) to force the server to evaluate and leak the contents of arbitrary local files.
Impact	This vulnerability allows an authenticated attacker to perform a Local File Read (LFR) attack against the underlying Linux file system. In the context of this specific engagement, the external SSH service enforced Two-Factor Authentication (2FA), effectively blocking access even with valid credentials. The exploitation of this BookStack vulnerability provides a critical bypass vector, allowing an attacker to read local configuration files, 2FA secrets, or private SSH keys directly from the server, thereby completely circumventing the perimeter authentication controls.
Remediation	1. **Apply Security Patches:** Update the BookStack installation to version 23.10.3 or a later stable release, which mitigates the SSRF vulnerability in the image handling logic. 2. **Input Validation:** Ensure that any application utilizing the `Intervention Image` library or similar components does not pass user-supplied URL data directly to object constructors. Input should be strictly validated, and temporary file paths should be used for image processing instead of direct remote/local URI loading.
References	-

Finding Evidence

Technical Details & Proof of Concept (PoC)

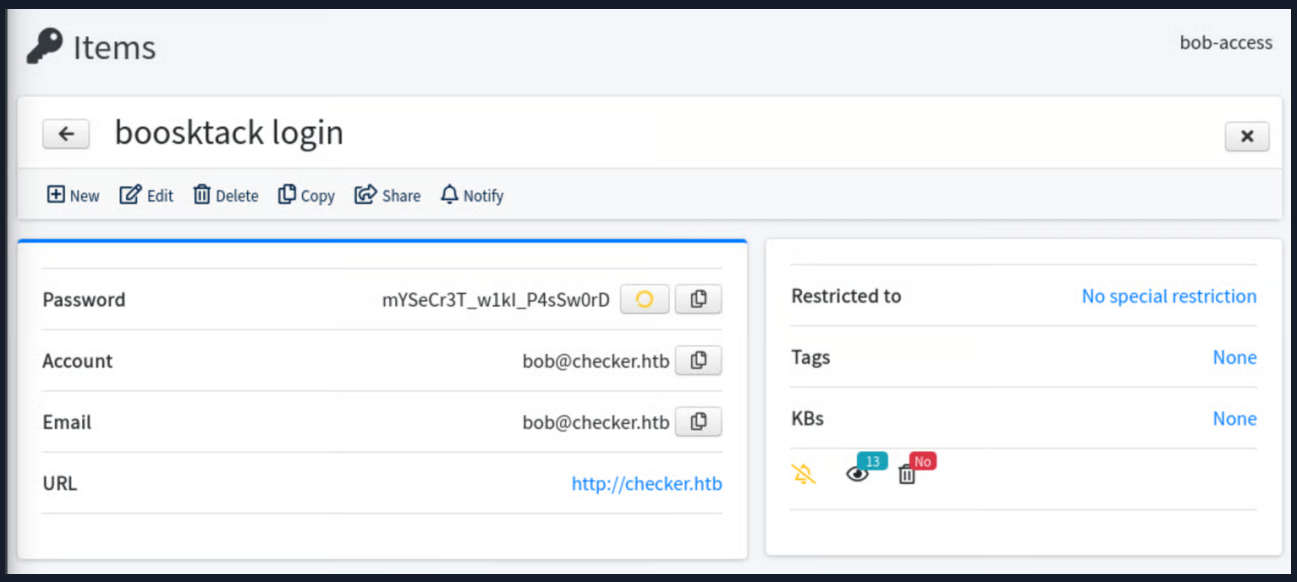
Step 1: Application Enumeration & Authentication

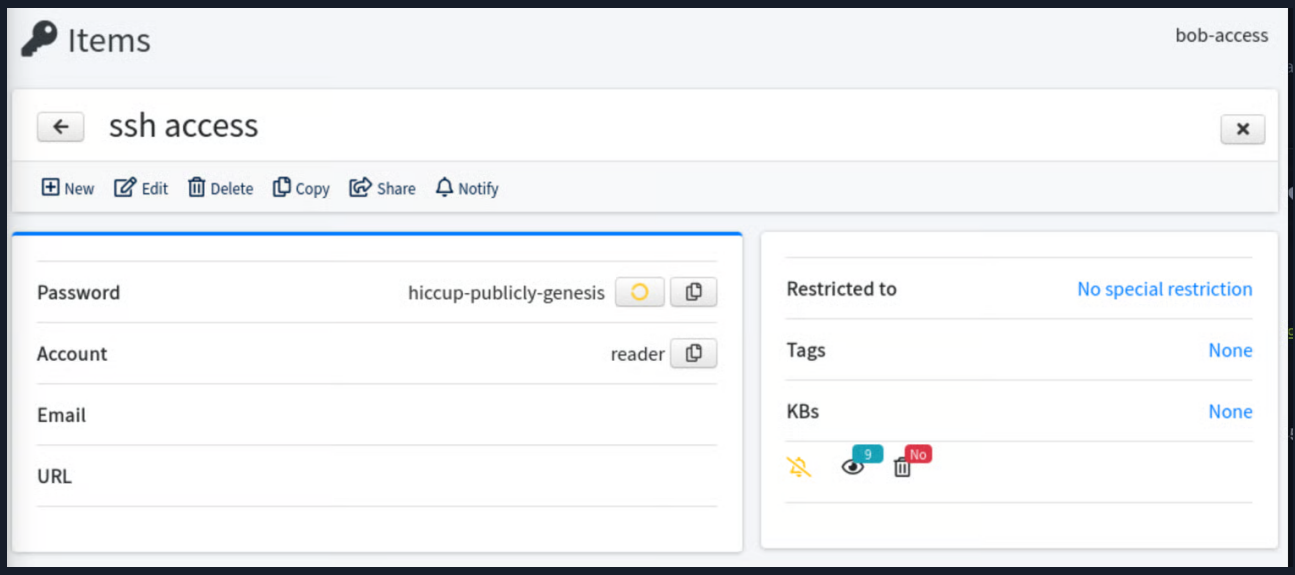
Accessing port 80 redirected the connection to `checker.htb`, which hosted a BookStack instance. Inspecting the page's source code explicitly disclosed the application version as `v23.10.2`. Using the credentials previously harvested from the compromised Teampass vault (`bob@checker.htb` : `mYSeCr3T_w1kI_P4sSw0rD`), successful authentication was achieved, granting the necessary privileges to interact with the vulnerable endpoints.



Step 2: 2FA Bypass Strategy Formulation

During concurrent testing, an attempt was made to access the system via SSH using the compromised `reader` credentials (`hiccup-publicly-genesis`). However, the server prompted for a verification code, indicating that Two-Factor Authentication was enabled. To bypass this, the BookStack SSRF vulnerability was targeted with the objective of reading the `reader` user's 2FA secret or SSH keys from the local file system.





Step 3: Exploit Preparation & Payload Modification

To exploit the Blind File Oracle technique via the SSRF, a public exploit repository (`synacktiv/php_filter_chains_oracle_exploit`) was cloned. Because the vulnerability lies within the `html` parameter and requires specific formatting to be processed by the image handling library, the exploit's `requestor.py` script required modification.

The Python script was updated to properly Base64 encode the generated PHP filter chains and wrap them within an HTML `<img>` tag.

Modifications made to `requestor.py`:

```
# First import base64
import base64

# Second, ensure the payload is base64 encoded and formatted as an image source
encoded_filter = base64.b64encode(filter_chain.encode('ascii')).decode('ascii')
final_payload = "<img+src='"+encoded_filter+"'/>"
merged_data = self.parse_parameter(final_payload)

# Third, update the session request to format the POST data correctly
requ = self.session.put(self.target, data='&'.join(f'{k}={v}' for k, v in
merged_data.items()))
```

By injecting this crafted `<img src...>` payload into the `html` parameter of the `/ajax/page/save-draft` POST request, the application is forced to process the PHP filter chain, leaking file contents piece by piece via error-based timing or response anomalies.

A Appendix

A.1 Finding Severities

Each finding has been assigned a severity rating of critical, high, medium, low or info. The rating is based off of an assessment of the priority with which each finding should be viewed and the potential impact each has on the confidentiality, integrity, and availability of checker.htb's data.

Rating	CVSS Score Range
Critical	9.0 – 10.0
High	7.0 – 8.9
Medium	4.0 – 6.9
Low	0.1 – 3.9
Info	0.0

A.2 Host & Service Discovery

IP Address	Port	Service	Notes
TODO FILL IN AS APPROPRIATE			

A.3 Subdomain Discovery

URL	Description	Discovery Method
TODO FILL IN DISCOVERED VHOSTS/SUBDOMAINS		

A.4 Exploited Hosts

Host	Scope	Method	Notes
TODO FILL IN AS APPROPRIATE	Text	Text	Text

A.5 Compromised Users

Username	Type	Method	Notes
TODO FILL IN AS APPROPRIATE	Text	Text	Text

A.6 Changes/Host Cleanup

Host	Scope	Change/Cleanup Needed
TODO FILL IN AS APPROPRIATE		

A.7 Flags Discovered

Flag #	Host	Flag Value	Flag Location	Method Used
1.	TODO HOSTNAME	TODO MD5 HASH	TODO Web root	TODO Unrestricted file upload (example)
2.				
3.				
4.				
5.				
6.				
7.				
8.				
9.				
10.				
11.				
12.				
13.				

End of Report

This report was rendered
by SysReptor with
♥